



BASES TECNOLOGICAS PARA EL SERVICIO PÚBLICO.

PROGRAMACIÓN CON PHP.

MODULO 5.

5B. Entrega completa y final. Práctica. Autenticación
utilizando OAuth 2.0.

ALUMNO: HURTADO LÓPEZ PEDRO.

PROFESOR: MEDINA PINZÓN ALEXIS GIOVANNI.

Miércoles 22 de octubre del 2025.

Introducción.

En esta práctica se desarrolla una API REST utilizando el framework de LARAVEL y levantando un contenedor en DOCKER, con el propósito de aplicar los fundamentos de BACKEND.

También se configura el entorno de trabajo, se crean modelos para gestionar datos mediante operaciones CRUD (Crear, Leer, Actualizar y Eliminar), y finalmente se realizan pruebas utilizando Postman para validar la correcta comunicación entre cliente y servidor.

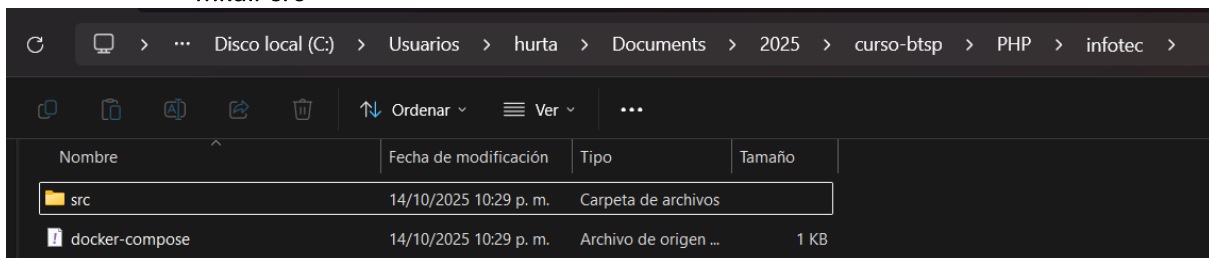
Esta práctica permite fortalecer los conocimientos en programación con PHP, manejo e introducción a APACHE y MySQL, así como la creación del contenedor en DOCKER.

Etaa 1. Preparación del entorno de trabajo

En esta etapa se prepara el entorno de trabajo para desarrollar la API REST utilizando Docker y Laravel.

1. Abrir una terminal PowerShell.
2. Crear las carpetas de trabajo con los comandos:

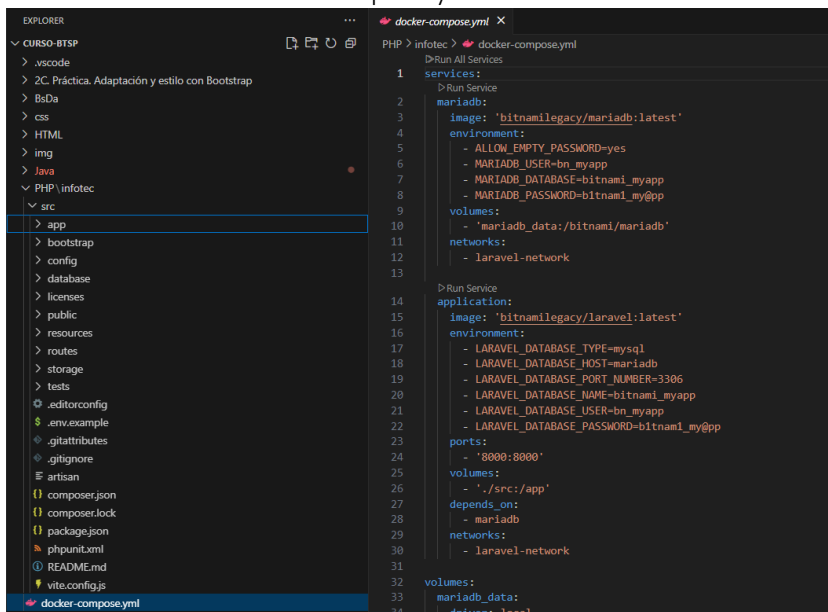
```
mkdir infotec && cd infotec
mkdir src
```



3. Abrir Visual Studio Code en el directorio infotec con el comando:

```
code .
```

4. Crear el archivo docker-compose.yml con el contenido indicado en la práctica.

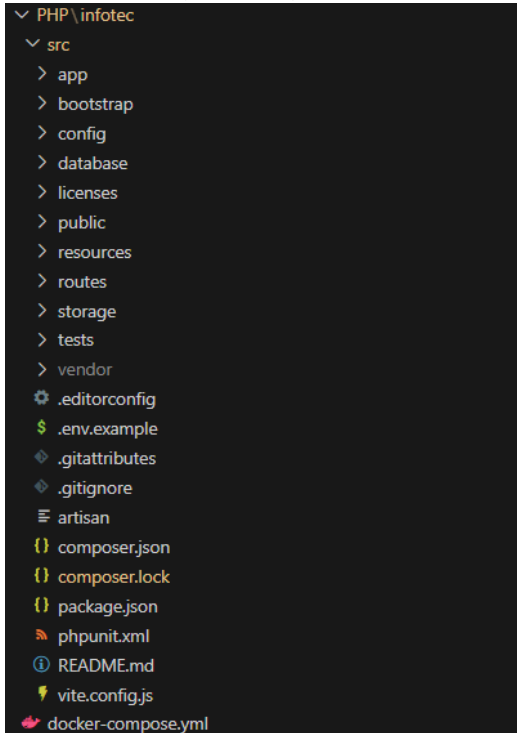


4. Ejecutar el comando:

docker compose up --build -d

```
PS C:\Users\hurta\Documents\2025\curso-btsp\PHP\infotec> docker compose up --build -d
[+] Running 4/4
  ✓ mariadb Pulled                                57.1s
  ✓ 0d29e489e89d Pull complete                    54.6s
  ✓ application Pulled                             112.7s
  ✓ f4e3fa1cd685 Pull complete                    110.3s
[+] Running 4/4
  ✓ Network infotec_laravel-network Created        0.1s
  ✓ Volume "infotec_mariadb_data" Created          0.0s
  ✓ Container infotec-mariadb-1 Started            1.3s
  ✓ Container infotec-application-1 Started        0.7s
```

5. Verificar que la carpeta src contiene el proyecto de Laravel.



6. Abrir <http://localhost:8000> y verificar la correcta instalación de Laravel.

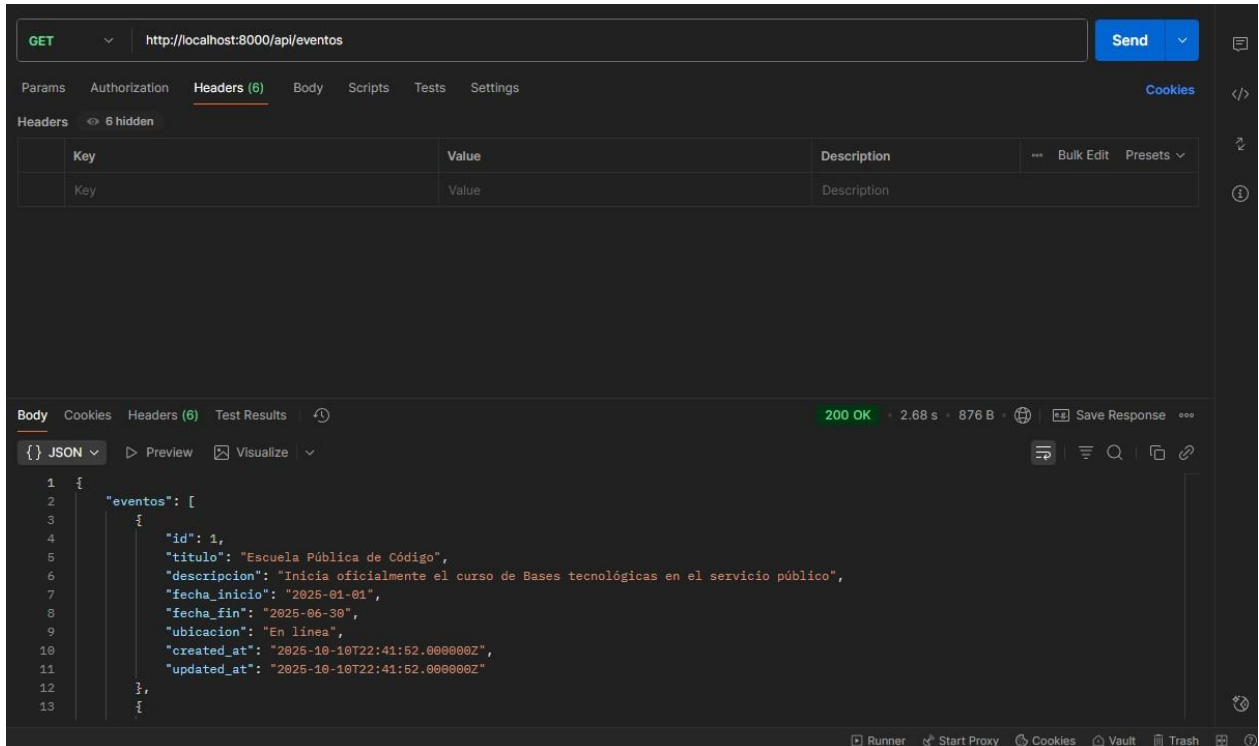
Inserte aquí captura de pantalla del sitio de Laravel funcionando en el navegador. Identificación de Clases y Objetos.

☐	▼	infotec	-	-	-	0.02%	0 seconds ago			
☐		mariadb-1	2acf73114b8f	bitnamilegacy/mariadb:lat		0.02%	1 second ago			
☐		application-1	cf933c11e498	bitnamilegacy/laravel:late: 8000.8000		0%	0 seconds ago			

Etaa 3. Probando la API REST

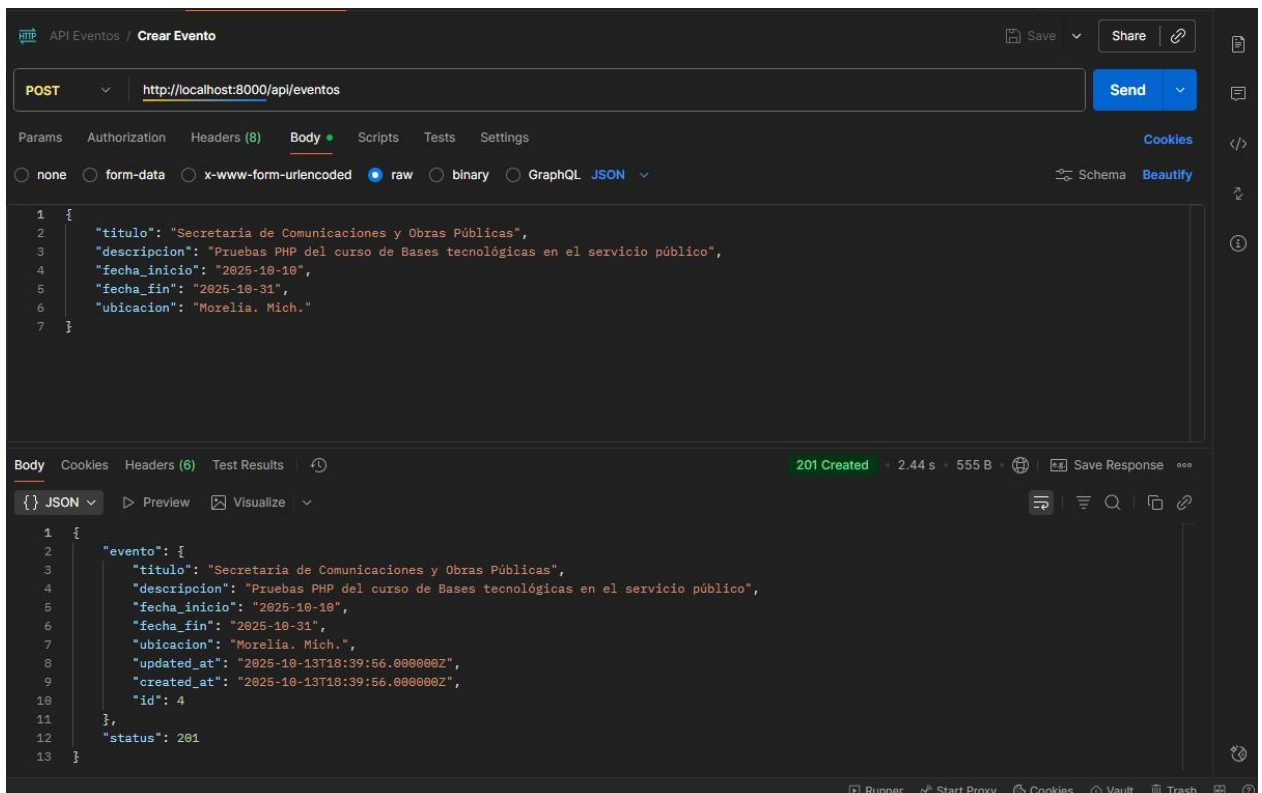
En esta etapa se utiliza Postman para validar el correcto funcionamiento de la API REST desarrollada.

1. Abrir la aplicación Postman.
2. Crear una nueva petición HTTP tipo GET hacia:
<http://localhost:8000/api/eventos>

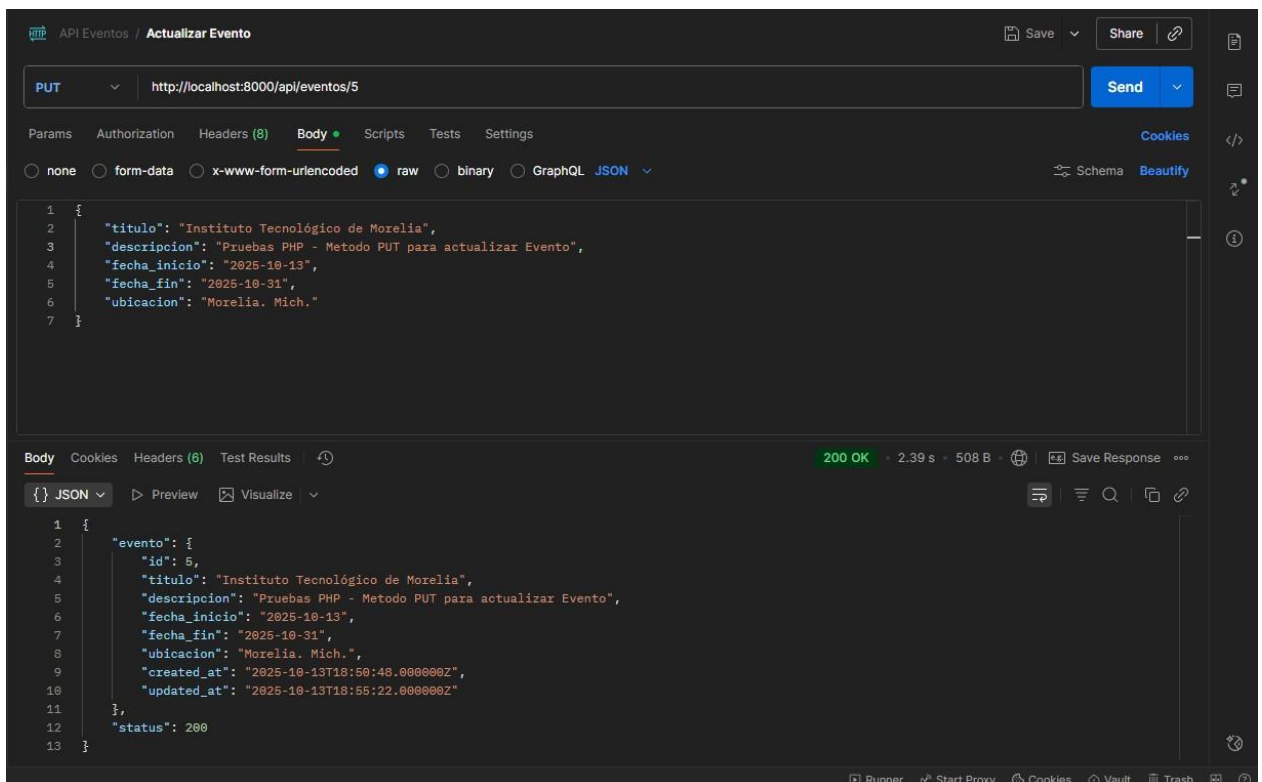


3. Crear una nueva petición HTTP tipo POST con el siguiente cuerpo JSON:

```
{
  "titulo": "Escuela Pública de Código",
  "descripcion": "Inicia oficialmente el curso de Bases tecnológicas en el servicio público",
  "fecha_inicio": "2025-01-01",
  "fecha_fin": "2025-06-30",
  "ubicacion": "En línea"
}
```



4. Actualizar un evento mediante una petición PUT modificando los valores del JSON.



5. Eliminar un evento mediante una petición DELETE:

<http://localhost:8000/api/eventos/{id}>

The screenshot shows the Postman interface for a DELETE request. The URL is `http://localhost:8000/api/eventos/3`. The response status is **404 Not Found** with a time of 1.11 s and a size of 249 B. The response body is in JSON format:

```
{
  "message": "Evento no encontrado",
  "status": 404
}
```

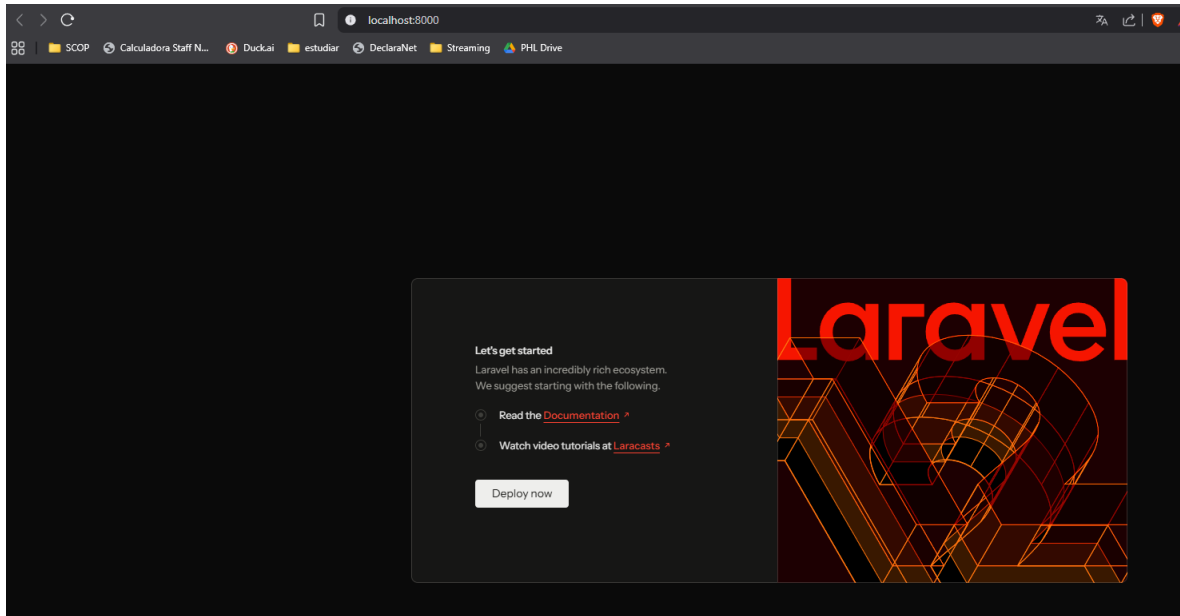
The screenshot shows the Postman interface for a DELETE request. The URL is `http://localhost:8000/api/eventos/4`. The response status is **200 OK** with a time of 2.19 s and a size of 238 B. The response body is in JSON format:

```
{
  "message": "Evento eliminado",
  "status": 200
}
```

Accede a la URL:

- <http://localhost:8000> en el navegador

LARAVEL.



Configuración del guard api → auth.php-

```
api.php  AsistenteController.php  PonenteController.php  Asistente.php  Ponente.php  Evento.php  .env  auth.php x
Curso BTSP > PHP > infotec > src > config > auth.php
1  <?php
2
3  return [
4
5      /*
6      |-----
7      | Authentication Defaults
8      |-----
9      |
10     | This option defines the default authentication "guard" and password
11     | reset "broker" for your application. You may change these values
12     | as required, but they're a perfect start for most applications.
13     |
14     | 'defaults' => [
15     |     'guard' => env('AUTH_GUARD', 'web'),
16     |     'passwords' => env('AUTH_PASSWORD_BROKER', 'users'),
17     | ],
18     */
19
20     'defaults' => [
21         'guard' => 'api',
22         'passwords' => env('AUTH_PASSWORD_BROKER', 'users'),
23     ],
24
25     /*
26     |-----
27     | Authentication Guards
28     |-----
29     |
30     | Next, you may define every authentication guard for your application.
31     | Of course, a great default configuration has been defined for you
32     | which utilizes session storage plus the Eloquent user provider.
33     |
34     | All authentication guards have a user provider, which defines how the
35     | users are actually retrieved out of your database or other storage
36     | system used by the application. Typically, Eloquent is utilized.
37     |
38     | Supported: "session"
39     |
40     */
41
42     'guards' => [
43         'web' => [
44             'driver' => 'session',
45             'provider' => 'users',
46         ],
47         'api' => [
48             'driver' => 'passport',
49             'provider' => 'users',
50         ],
51     ],
52 ];
```

CÓDIGO en VSCode.

```
<?php

return [

    /*
    |-----
    | Authentication Defaults
    |-----
    |
    | This option defines the default authentication "guard" and password
    | reset "broker" for your application. You may change these values
    | as required, but they're a perfect start for most applications.
    |
    'defaults' => [
        'guard' => env('AUTH_GUARD', 'web'),
        'passwords' => env('AUTH_PASSWORD_BROKER', 'users'),
    ],
    */

    'defaults' => [
        'guard' => 'api',
        'passwords' => env('AUTH_PASSWORD_BROKER', 'users'),
    ],

    /*
    |-----
    | Authentication Guards
    |-----
    |
    | Next, you may define every authentication guard for your application.
    | Of course, a great default configuration has been defined for you
    | which utilizes session storage plus the Eloquent user provider.
    |
    | All authentication guards have a user provider, which defines how the
    | users are actually retrieved out of your database or other storage
    | system used by the application. Typically, Eloquent is utilized.
    |
    | Supported: "session"
    |
    */

    'guards' => [
        'web' => [
            'driver' => 'session',
            'provider' => 'users',
```



```

    ],
    'api' => [
        'driver' => 'keycloak', // usa el guard de Keycloak
        'provider' => 'users',
    ],
],

/*
|-----
--
| User Providers
|-----
--
|
| All authentication guards have a user provider, which defines how the
| users are actually retrieved out of your database or other storage
| system used by the application. Typically, Eloquent is utilized.
|
| If you have multiple user tables or models you may configure multiple
| providers to represent the model / table. These providers may then
| be assigned to any extra authentication guards you have defined.
|
| Supported: "database", "eloquent"
|
*/

'providers' => [
    'users' => [
        'driver' => 'eloquent',
        'model' => env('AUTH_MODEL', App\Models\User::class),
    ],

    // 'users' => [
    //     'driver' => 'database',
    //     'table' => 'users',
    // ],
],

/*
|-----
--
| Resetting Passwords
|-----
--
|
| These configuration options specify the behavior of Laravel's password
| reset functionality, including the table utilized for token storage
| and the user provider that is invoked to actually retrieve users.
|
| The expiry time is the number of minutes that each reset token will be
| considered valid. This security feature keeps tokens short-lived so

```

```

| they have less time to be guessed. You may change this as needed.
|
| The throttle setting is the number of seconds a user must wait before
| generating more password reset tokens. This prevents the user from
| quickly generating a very large amount of password reset tokens.
|
*/

'passwords' => [
  'users' => [
    'provider' => 'users',
    'table' => env('AUTH_PASSWORD_RESET_TOKEN_TABLE',
'password_reset_tokens'),
    'expire' => 60,
    'throttle' => 60,
  ],
],

/*
|-----
--
| Password Confirmation Timeout
|-----
--
|
| Here you may define the number of seconds before a password
confirmation
| window expires and users are asked to re-enter their password via the
| confirmation screen. By default, the timeout lasts for three hours.
|
*/

'password_timeout' => env('AUTH_PASSWORD_TIMEOUT', 10800),

];

```

Protección de endpoints dentro de → `Route::middleware('auth.api')`.

```
api.php x AsistenteController.php PonenteController.php Asistente.php Ponente.php Evento.php .env auth.php
Curso BTSP > PHP > infotec > src > routes > api.php
1 <?php
2
3 use Illuminate\Http\Request;
4 use Illuminate\Support\Facades\Route;
5 // Agregar el controlador EventoController
6 use App\Http\Controllers\EventoController;
7 // Agregar el controlador PonenteController
8 use App\Http\Controllers\PonenteController;
9 // Agregar el controlador AsistenteController
10 use App\Http\Controllers\AsistenteController;
11
12
13 /**
14  * Rutas públicas
15  */
16 // Recuperar todos los eventos
17 Route::get('/eventos', [EventoController::class, 'index']);
18 // Recuperar un evento específico
19 Route::get('/eventos/{id}', [EventoController::class, 'show']);
20 // Recuperar todos los ponentes
21 Route::get('/ponentes', [PonenteController::class, 'index']);
22 // Recuperar un ponente específico
23 Route::get('/ponentes/{id}', [PonenteController::class, 'show']);
24
25
26 /**
27  * Rutas Privadas
28  */
29 Route::middleware('auth:api')->group(function () {
30     // Almacenar un evento nuevo
31     Route::post('/eventos', [EventoController::class, 'store']);
32     // Actualizar un evento específico
33     Route::put('/eventos/{evento}', [EventoController::class, 'update']);
34     // Eliminar un evento específico
35     Route::delete('/eventos/{id}', [EventoController::class, 'destroy']);
36
37     // Almacenar un ponente nuevo
38     Route::post('/ponentes', [PonenteController::class, 'store']);
39     // Actualizar un ponente específico
40     Route::put('/ponentes/{ponente}', [PonenteController::class, 'update']);
41     // Eliminar un ponente específico
42     Route::delete('/ponentes/{id}', [PonenteController::class, 'destroy']);
43
44     // Recuperar todos los asistentes
45     Route::get('/asistentes', [AsistenteController::class, 'index']);
46     // Almacenar un asistente nuevo
47     Route::post('/asistentes', [AsistenteController::class, 'store']);
48     // Recuperar un asistente específico
49     Route::get('/asistentes/{id}', [AsistenteController::class, 'show']);
50     // Actualizar un asistente específico
51     Route::put('/asistentes/{asistente}', [AsistenteController::class, 'update']);
52     // Eliminar un asistente específico
53     Route::delete('/asistentes/{id}', [AsistenteController::class, 'destroy']);
54 });
```

CÓDIGO en VSCode.

```
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
// Agregar el controlador EventoController
use App\Http\Controllers\EventoController;
// Agregar el controlador PonenteController
use App\Http\Controllers\PonenteController;
// Agregar el controlador AsistenteController
use App\Http\Controllers\AsistenteController;

/**
 * Rutas públicas
 */
// Recuperar todos los eventos
Route::get('/eventos', [EventoController::class, 'index']);
// Recuperar un evento específico
Route::get('/eventos/{id}', [EventoController::class, 'show']);
```

```
// Recuperar todos los ponentes
Route::get('/ponentes', [PonenteController::class, 'index']);
// Recuperar un ponente específico
Route::get('/ponentes/{id}', [PonenteController::class, 'show']);

/**
 * Rutas Privadas
 */
Route::middleware('auth:api')->group(function () {
    // Almacenar un evento nuevo
    Route::post('/eventos', [EventoController::class, 'store']);
    // Actualizar un evento específico
    Route::put('/eventos/{evento}', [EventoController::class, 'update']);
    // Eliminar un evento específico
    Route::delete('/eventos/{id}', [EventoController::class, 'destroy']);

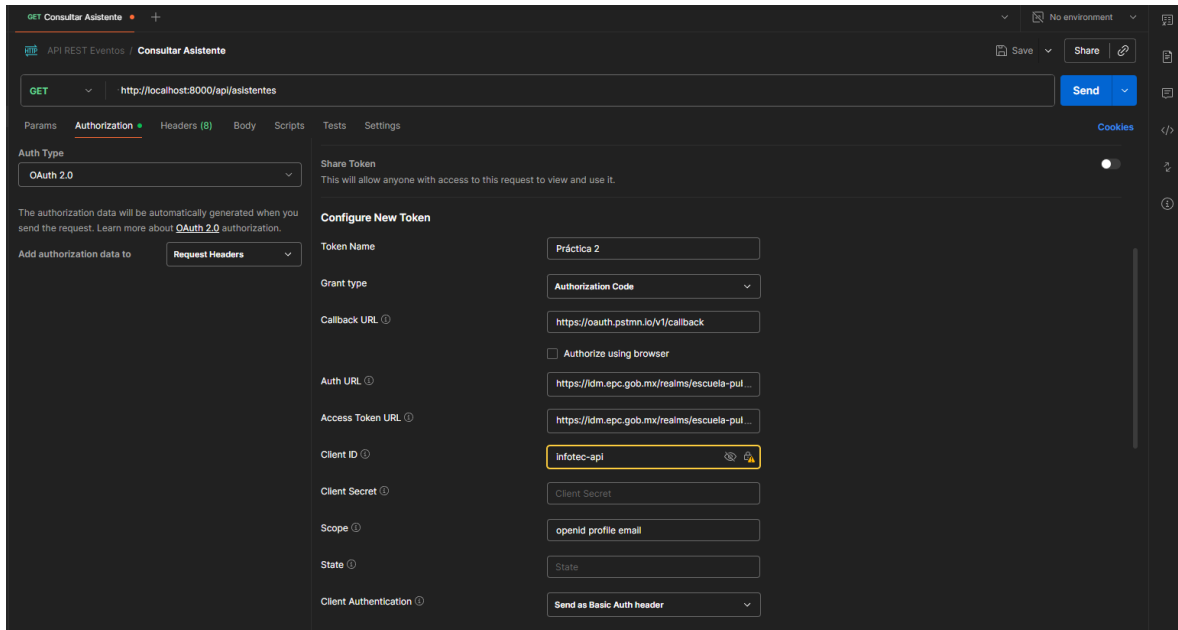
    // Almacenar un ponente nuevo
    Route::post('/ponentes', [PonenteController::class, 'store']);
    // Actualizar un ponente específico
    Route::put('/ponentes/{ponente}', [PonenteController::class, 'update']);
    // Eliminar un ponente específico
    Route::delete('/ponentes/{id}', [PonenteController::class, 'destroy']);

    // Recuperar todos los asistentes
    Route::get('/asistentes', [AsistenteController::class, 'index']);
    // Almacenar un asistente nuevo
    Route::post('/asistentes', [AsistenteController::class, 'store']);
    // Recuperar un asistente específico
    Route::get('/asistentes/{id}', [AsistenteController::class, 'show']);
    // Actualizar un asistente específico
    Route::put('/asistentes/{asistente}',
[AsistenteController::class, 'update']);
    // Eliminar un asistente específico
    Route::delete('/asistentes/{id}',
[AsistenteController::class, 'destroy']);
});
```

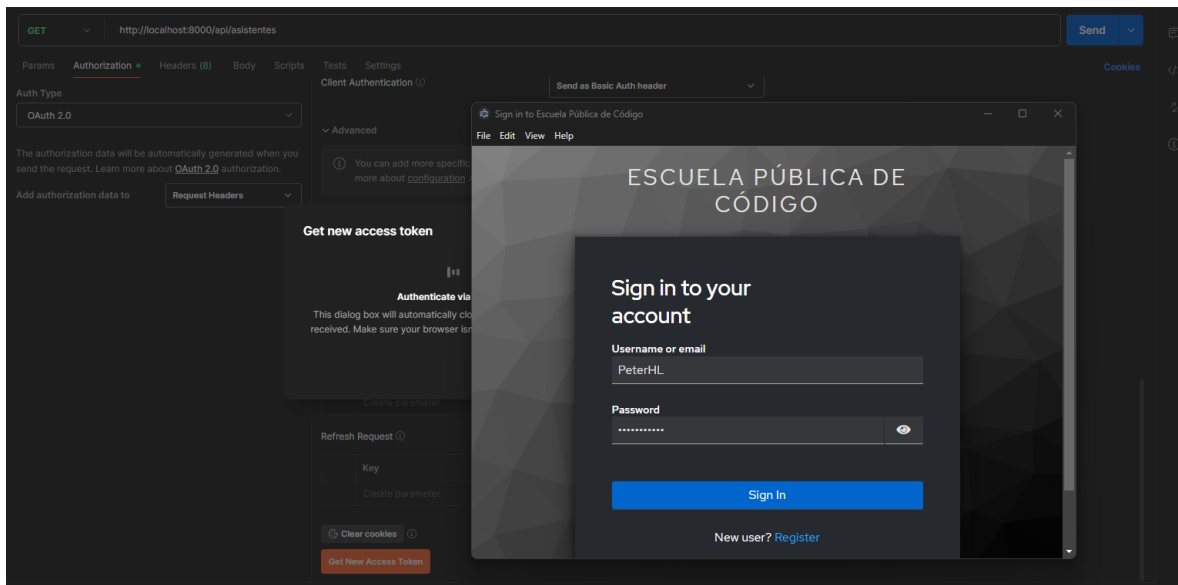
Contenedor en → DOCKER.

☐	▼	●	infotec	-	-	-	0.06%	8 minut	■	:	🗑
☐		●	application-1	40f34a31ec22	bitnamileg: 8000:8000	↗	0.05%	8 minut	■	:	🗑
☐		●	mariadb-1	f550c2093e90	bitnamileg:		0.01%	8 minut	■	:	🗑

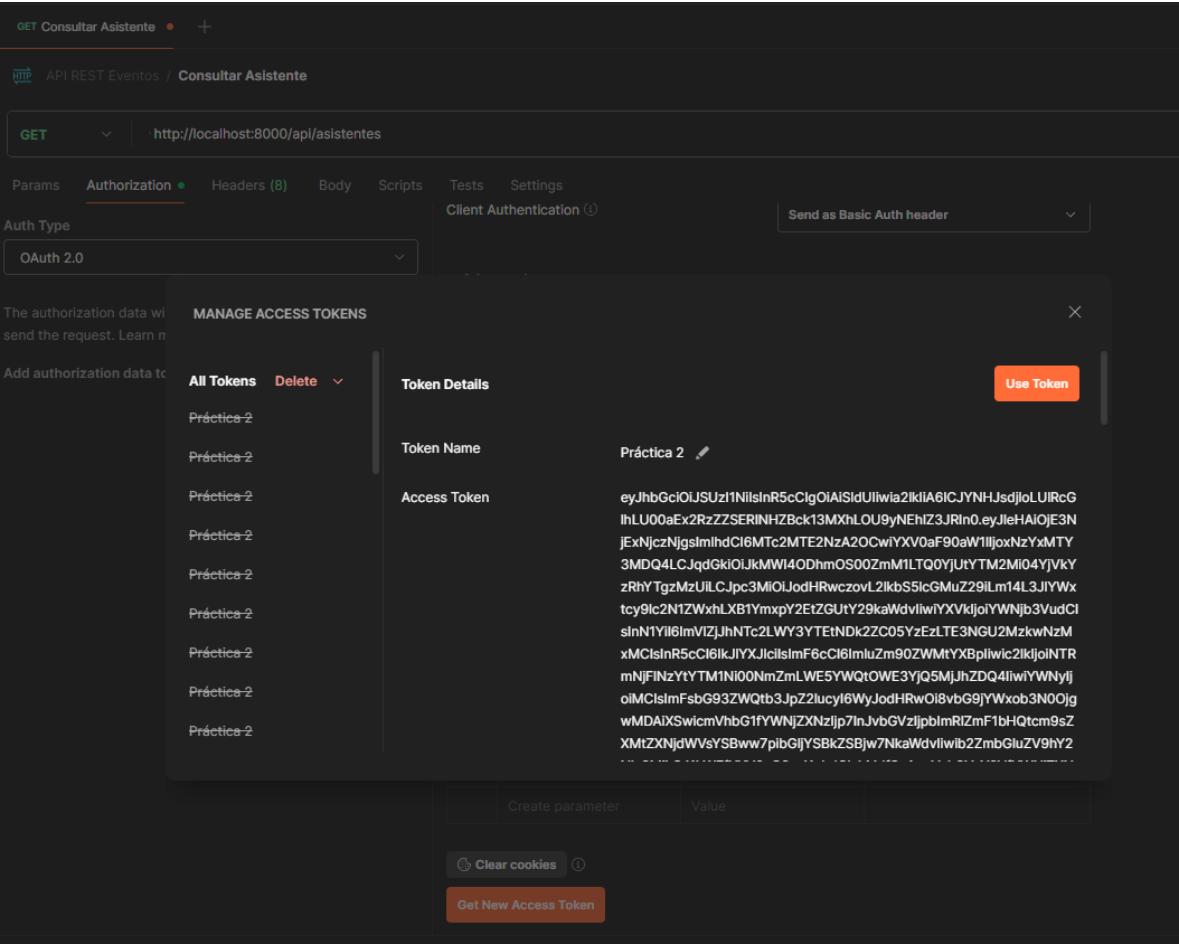
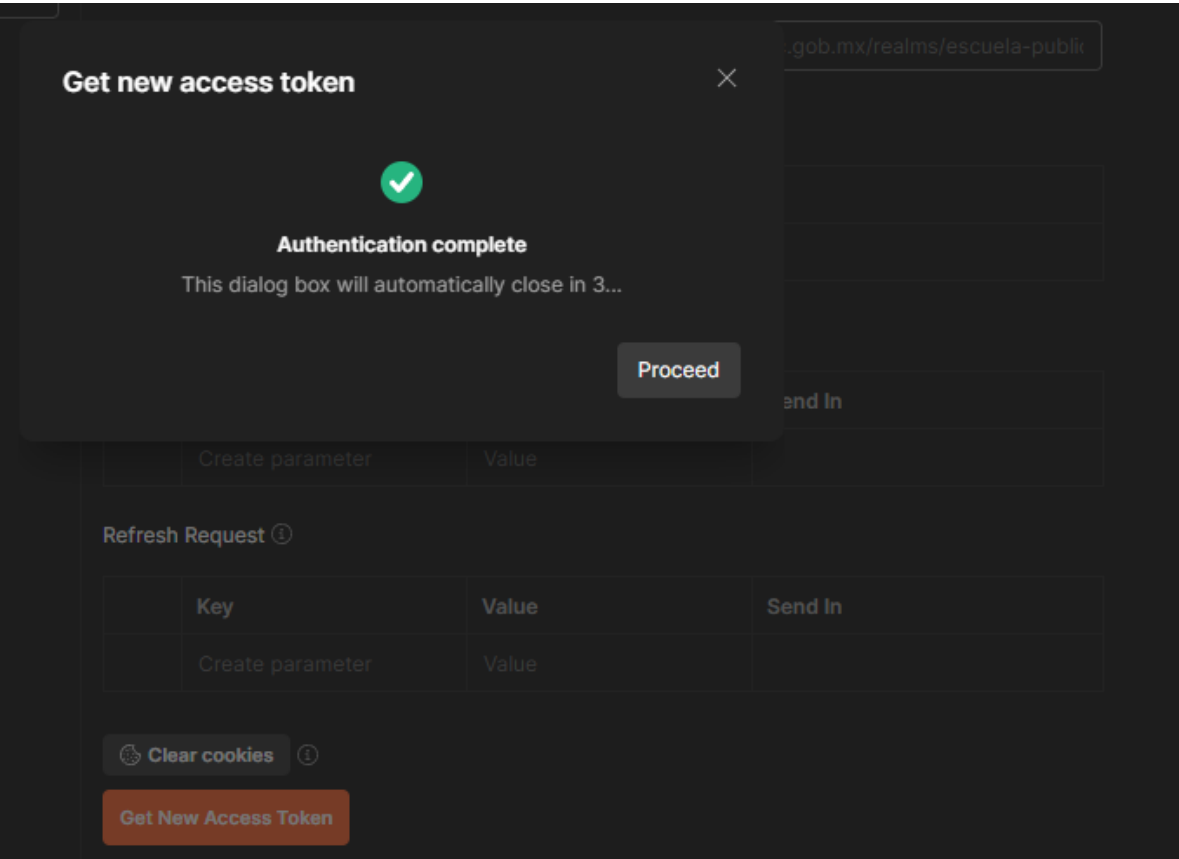
Configuración de los parámetros Authorization → Postman.

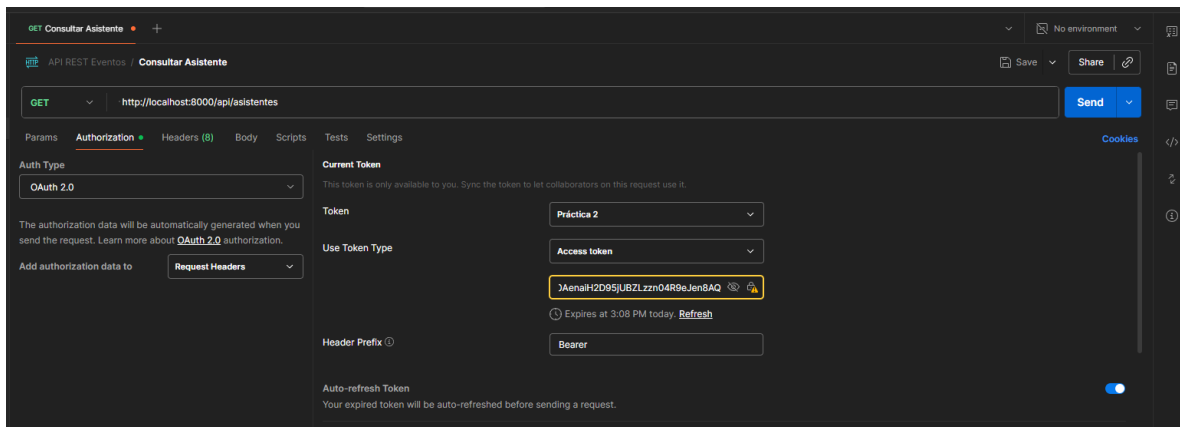


Creación de Usuario nuevo → Keycloak.

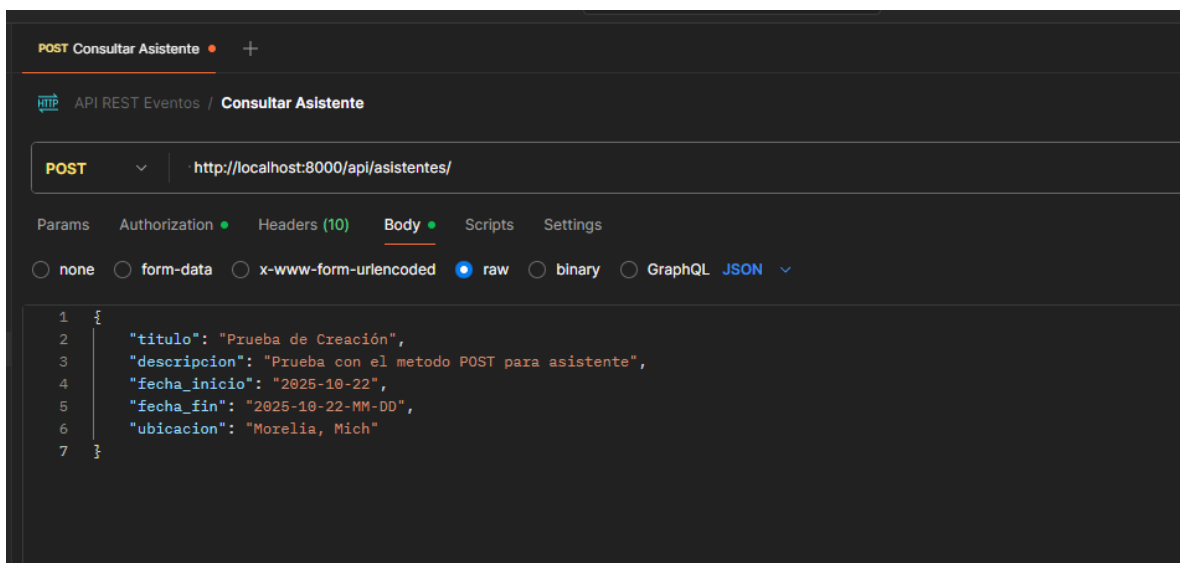
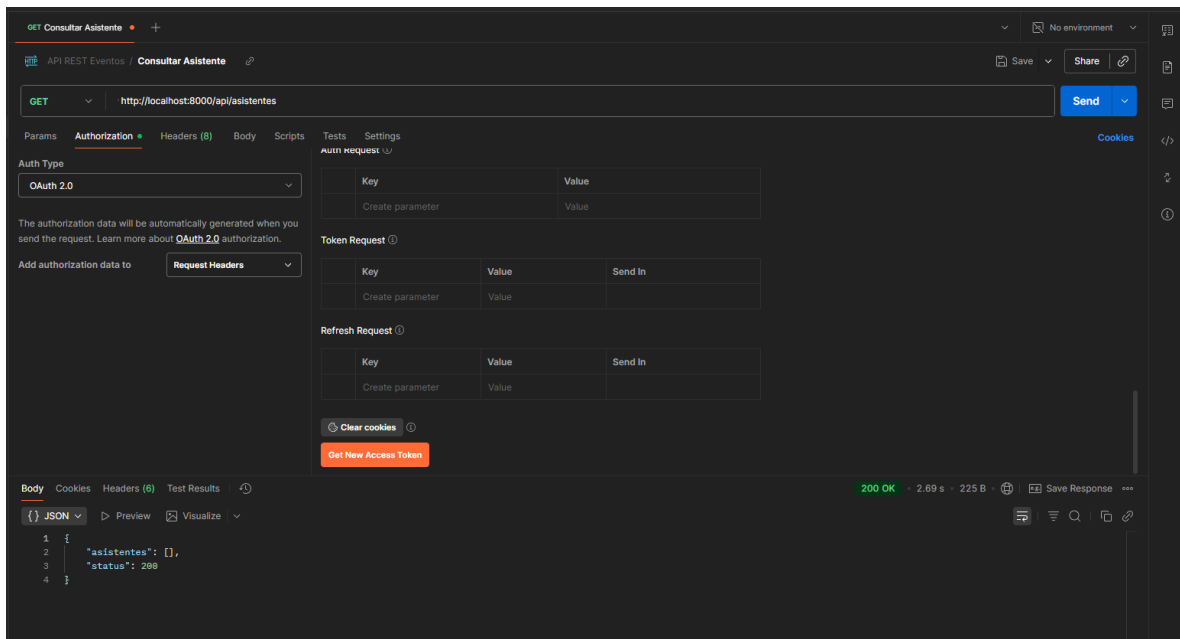


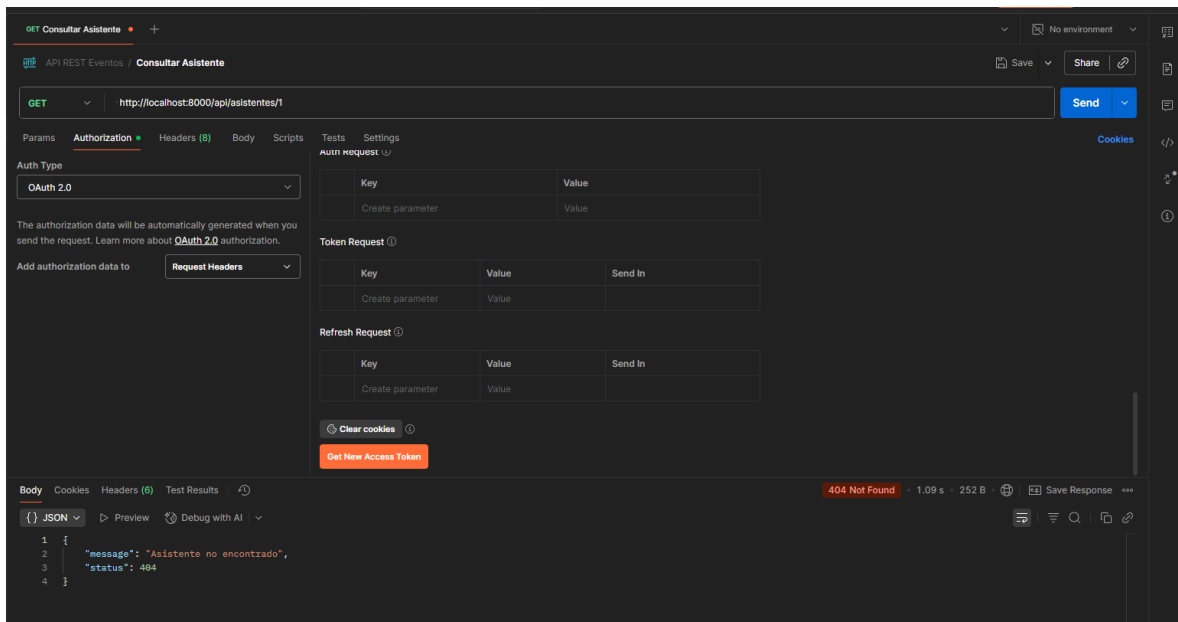
TOKEN → Obtención del Token autorización.





ACCESO → Endpoints privados (código 200, 201, 204).





Conclusiones.

- La práctica de autenticación de OAuth 2.0, logra una integración efectiva entre Laravel y Keycloak para asegurar el acceso a la API REST. Las configuraciones realizadas demuestran su eficacia al proteger los recursos y permitir el acceso únicamente a usuarios autenticados o permitidos.
- El uso de Postman permitió validar el funcionamiento adecuado de los mecanismos de autenticación y control de acceso, con la creación de los eventos GET, POST, PUT, DELETE, se prueba las entidades de los eventos.
- Se fortalecieron los conocimientos en seguridad, autenticación y autorización para el desarrollo de APIs.
- Se aprendió a configurar la Keycloak, la autorización del TOKEN mediante los parámetros establecidos en la práctica.
- Por último, se aprendió a sincronizar el servidor de LARAVEL con el servidor del KEYCLOAK, ya que no lo estaban y al momento de utilizar el TOKEN lanzaba un mensaje de error pero esto se soluciono con la instrucción `KEYCLOAK_LEEWAY = 60`, la cual se agrega en el archivo `.env`, después de ello el TOKEN es aceptado y funciono todo perfecto.